Analyse des Logs Firewall

Période analysée : Novembre 2025 – Février 2026

In [1]:
```
import re
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns

%matplotlib inline
plt.rcParams['figure.dpi'] = 120

INPUT_LOG = r"C:\Users\Thibaud\Desktop\challenge-2026\challenge-2026\données\log

TCP_PORTS = {
    20, 21, 22, 23, 25, 53, 80, 110, 143, 389, 443,
    445, 465, 587, 993, 995, 1433, 1521, 3306, 3389,
    5432, 5900, 8080, 8443, 8888
}
UDP_PORTS = {53, 67, 68, 69, 123, 161, 162, 500, 514, 520, 1194, 1900, 4500, 535

PALETTE = ["#2E75B6", "#ED7D31", "#70AD47", "#FFC000", "#7030A0", "#FF0000"]
sns.set_theme(style="whitegrid", palette=PALETTE)

PATTERN_DATE = re.compile(r'(Nov|Dec|Déc|Jan|Feb|Fév)')
print("✅ Configuration chargée")
```

```
✅ Configuration chargée
```

Chargement et parsing du log brut

In [2]:
```
rows = []
total = 0
skipped = 0

with open(INPUT_LOG, "r", errors="replace") as f:
    for line in f:
        total += 1
        if not PATTERN_DATE.search(line):
            continue
        try:
            src_ip   = re.search(r'SRC=([\d\.]+)',  line).group(1)
            dst_ip   = re.search(r'DST=([\d\.]+)',  line).group(1)
            proto    = re.search(r'PROTO=(\w+)',    line).group(1)
            dst_port = int(re.search(r'DPT=(\d+)', line).group(1))
            rule_id  = re.search(r'RULE=(\d+)',    line).group(1)
            action   = "PERMIT" if "PERMIT" in line else "DENY"
            date_match = re.search(r'^([A-Z][a-z]{2}\s+\d+\s+\d+:\d+:\d+)', line
            date = date_match.group(1) if date_match else "NA"
            rows.append({"timestamp": date, "src_ip": src_ip, "dst_ip": dst_ip,
                         "protocole": proto, "dst_port": dst_port, "action": act
        except Exception:
            skipped += 1
        if total % 1_000_000 == 0:
```

```
            print(f"  Lignes lues : {total:,} | Extraites : {len(rows):,}")

print(f"✅ {len(rows):,} lignes extraites sur {total:,} ({skipped:,} ignorées)"
```

```
  Lignes lues : 8,000,000 | Extraites : 575,251
  Lignes lues : 9,000,000 | Extraites : 1,575,251
  Lignes lues : 10,000,000 | Extraites : 2,575,251
  Lignes lues : 11,000,000 | Extraites : 3,575,251
✅ 4,572,903 lignes extraites sur 11,997,652 (0 ignorées)
```

Nettoyage et déduction des protocoles

In [3]:

```
df = pd.DataFrame(rows)

df["timestamp"] = pd.to_datetime(
    "2025 " + df["timestamp"].str.strip(),
    format="%Y %b %d %H:%M:%S", errors="coerce"
)
# Janvier et Février → 2026
df.loc[df["timestamp"].dt.month.isin([1, 2]), "timestamp"] += pd.DateOffset(year
df["dst_port"] = pd.to_numeric(df["dst_port"], errors="coerce")

def deduce_protocol(row):
    proto = str(row["protocole"]).strip().upper()
    if proto == "TCP":  return "TCP"
    if proto == "UDP":  return "UDP"
    if proto == "ICMP": return "ICMP"
    dport = row["dst_port"]
    if pd.notna(dport):
        if dport in TCP_PORTS: return "TCP"
        if dport in UDP_PORTS: return "UDP"
    return "AUTRE" if proto in ("", "N/A", "NA", "NONE", "0") else proto

df["proto_deduit"] = df.apply(deduce_protocol, axis=1)

print("Répartition des protocoles déduits :")
print(df["proto_deduit"].value_counts())
print(f"\nPériode : {df['timestamp'].min()} → {df['timestamp'].max()}")
df.head()
```

```
Répartition des protocoles déduits :
proto_deduit
TCP    4572903
Name: count, dtype: int64

Période : 2025-11-12 09:56:37 → 2026-02-12 10:08:39
```

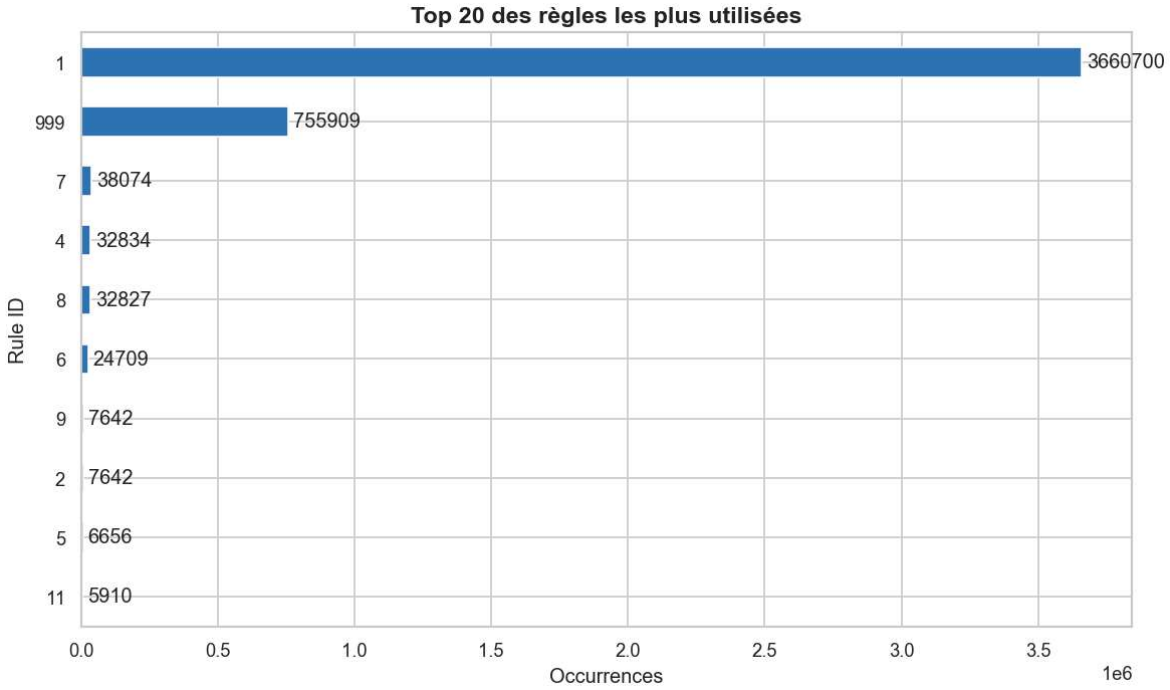
Out[3]:

	timestamp	src_ip	dst_ip	protocole	dst_port	action	rule_id	proto_
0	2025-11-12 09:56:37	77.90.185.64	159.84.146.99	TCP	65153	DENY	999	
1	2025-11-12 09:56:37	47.128.20.252	159.84.146.99	TCP	443	PERMIT	1	
2	2025-11-12 09:56:38	23.22.35.162	159.84.146.99	TCP	443	PERMIT	1	
3	2025-11-12 09:56:39	79.124.60.150	159.84.146.99	TCP	56283	DENY	999	
4	2025-11-12 09:56:39	47.128.20.252	159.84.146.99	TCP	443	PERMIT	1	

Chapitre 1 – Classement des règles les plus utilisées

In [4]:

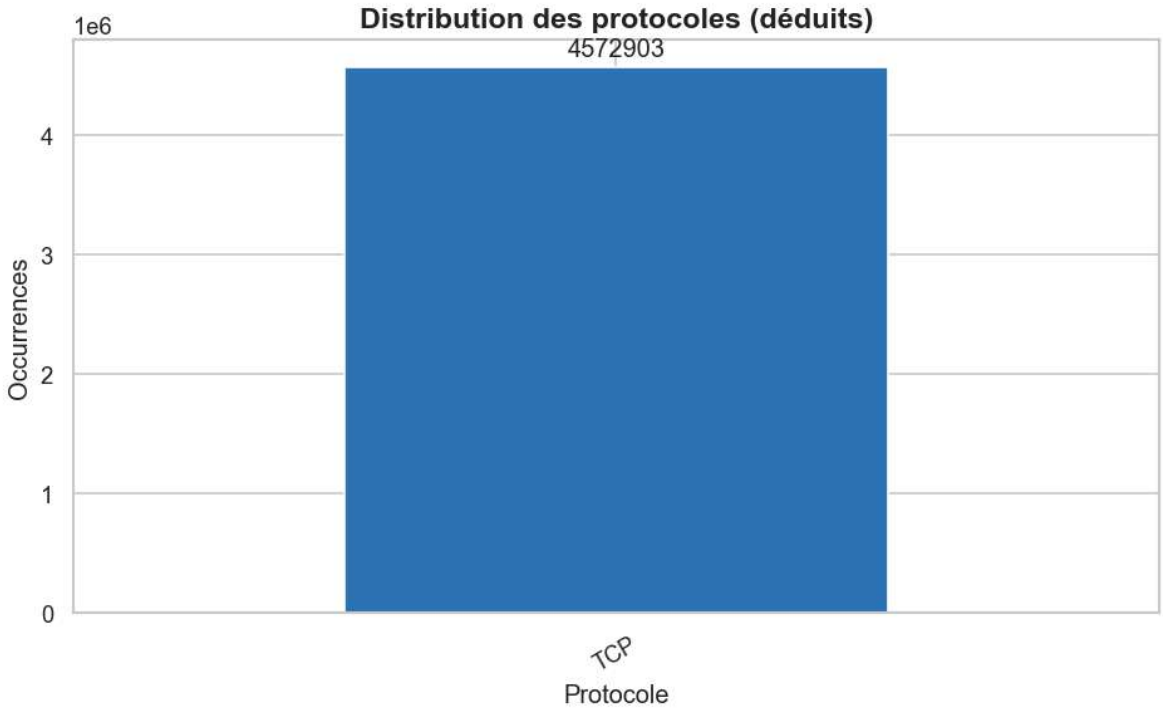
```
rule_counts = df["rule_id"].value_counts().head(20)

fig, ax = plt.subplots(figsize=(10, 6))
rule_counts.plot(kind="barh", ax=ax, color=PALETTE[0])
ax.invert_yaxis()
ax.set_title("Top 20 des règles les plus utilisées", fontsize=14, fontweight="bo
ax.set_xlabel("Occurrences")
ax.set_ylabel("Rule ID")
ax.bar_label(ax.containers[0], fmt="%d", padding=3)
plt.tight_layout()
plt.show()

print("\n📋 Tableau Top 10 :")
print(rule_counts.head(10).rename("Occurrences").to_frame().to_string())
```



```
📋 Tableau Top 10 :
         Occurrences
rule_id
1            3660700
999           755909
7              38074
4              32834
8              32827
6              24709
9               7642
2               7642
5               6656
11              5910
```

Chapitre 2 – Distribution des protocoles

In [5]:
```
proto_counts = df["proto_deduit"].value_counts()

fig, ax = plt.subplots(figsize=(8, 5))
proto_counts.plot(kind="bar", ax=ax, color=PALETTE[:len(proto_counts)], edgecolo
ax.set_title("Distribution des protocoles (déduits)", fontsize=14, fontweight="b
ax.set_xlabel("Protocole")
ax.set_ylabel("Occurrences")
ax.tick_params(axis="x", rotation=30)
ax.bar_label(ax.containers[0], fmt="%d", padding=3)
plt.tight_layout()
plt.show()

print("\nAucun log UDP détecté — chapitre UDP passé.")
```




```
Aucun log UDP détecté — chapitre UDP passé.
```

Chapitre 3 – Top 10 des règles les plus utilisées (UDP)

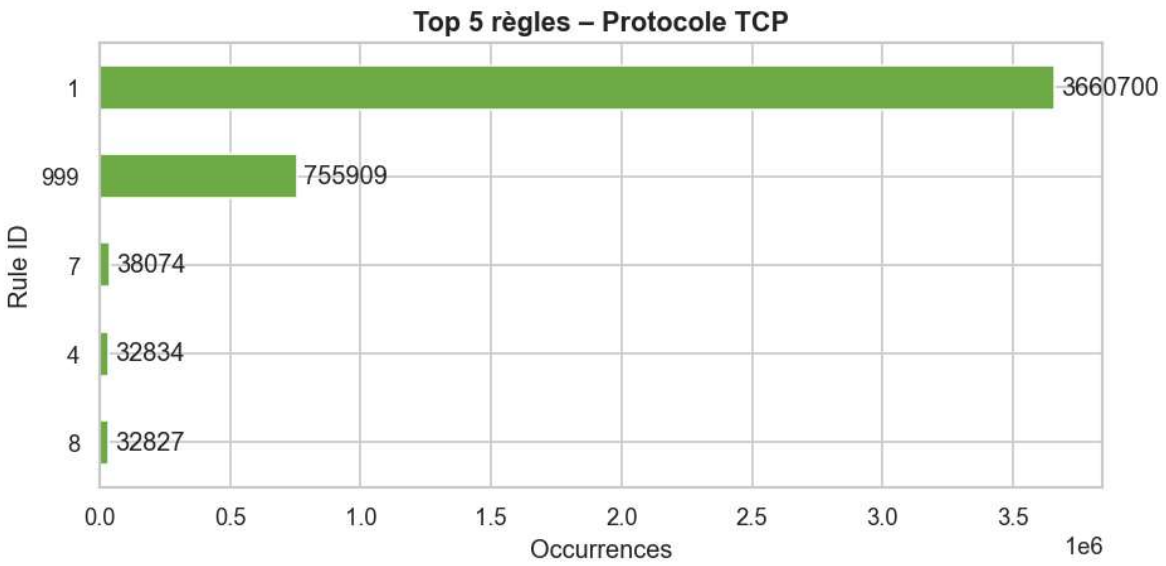
> ⚠️ Aucun log UDP trouvé dans les données — chapitre non applicable.

In [6]:
```
df_udp = df[df["proto_deduit"] == "UDP"]

if df_udp.empty:
    print("ℹ️ Aucun événement UDP dans la période analysée. Chapitre non applica
else:
    udp_rules = df_udp["rule_id"].value_counts().head(10)
    fig, ax = plt.subplots(figsize=(9, 5))
    udp_rules.plot(kind="barh", ax=ax, color=PALETTE[1])
    ax.invert_yaxis()
    ax.set_title("Top 10 règles – Protocole UDP", fontsize=13, fontweight="bold"
    ax.set_xlabel("Occurrences")
    ax.set_ylabel("Rule ID")
    ax.bar_label(ax.containers[0], fmt="%d", padding=3)
    plt.tight_layout()
    plt.show()
```

```
ℹ️ Aucun événement UDP dans la période analysée. Chapitre non applicable.
```

Chapitre 4 – Top 5 des règles les plus utilisées (TCP)

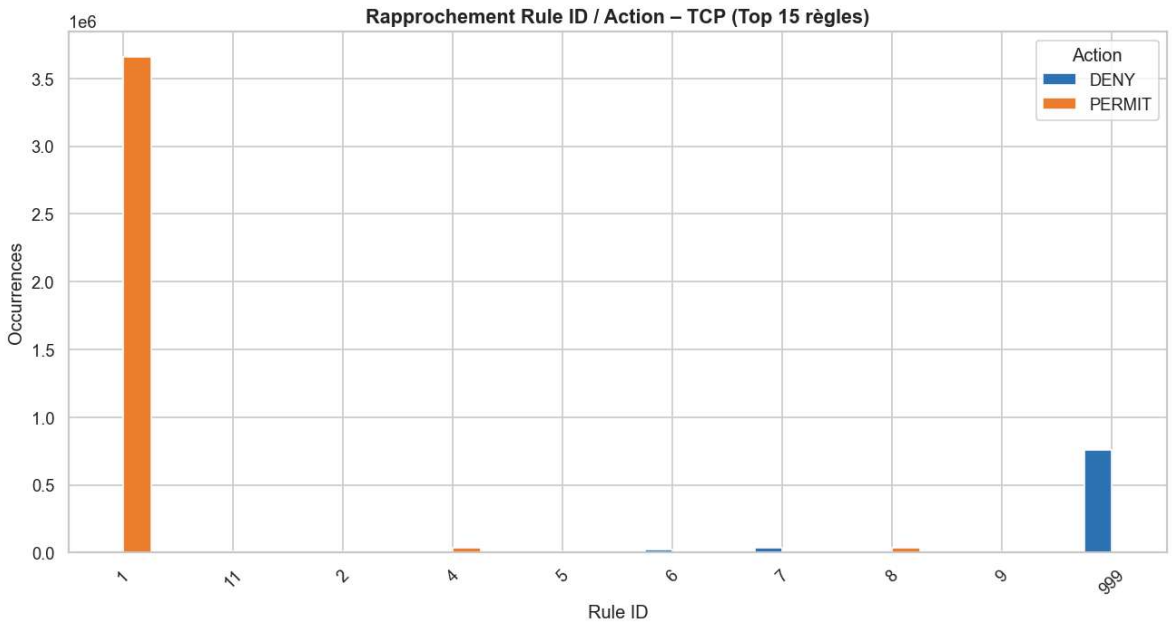
In [7]:
```
df_tcp = df[df["proto_deduit"] == "TCP"]
tcp_rules = df_tcp["rule_id"].value_counts().head(5)

fig, ax = plt.subplots(figsize=(8, 4))
tcp_rules.plot(kind="barh", ax=ax, color=PALETTE[2])
ax.invert_yaxis()
ax.set_title("Top 5 règles – Protocole TCP", fontsize=13, fontweight="bold")
ax.set_xlabel("Occurrences")
ax.set_ylabel("Rule ID")
ax.bar_label(ax.containers[0], fmt="%d", padding=3)
plt.tight_layout()
plt.show()

print(tcp_rules.rename("Occurrences TCP").to_frame().to_string())
```



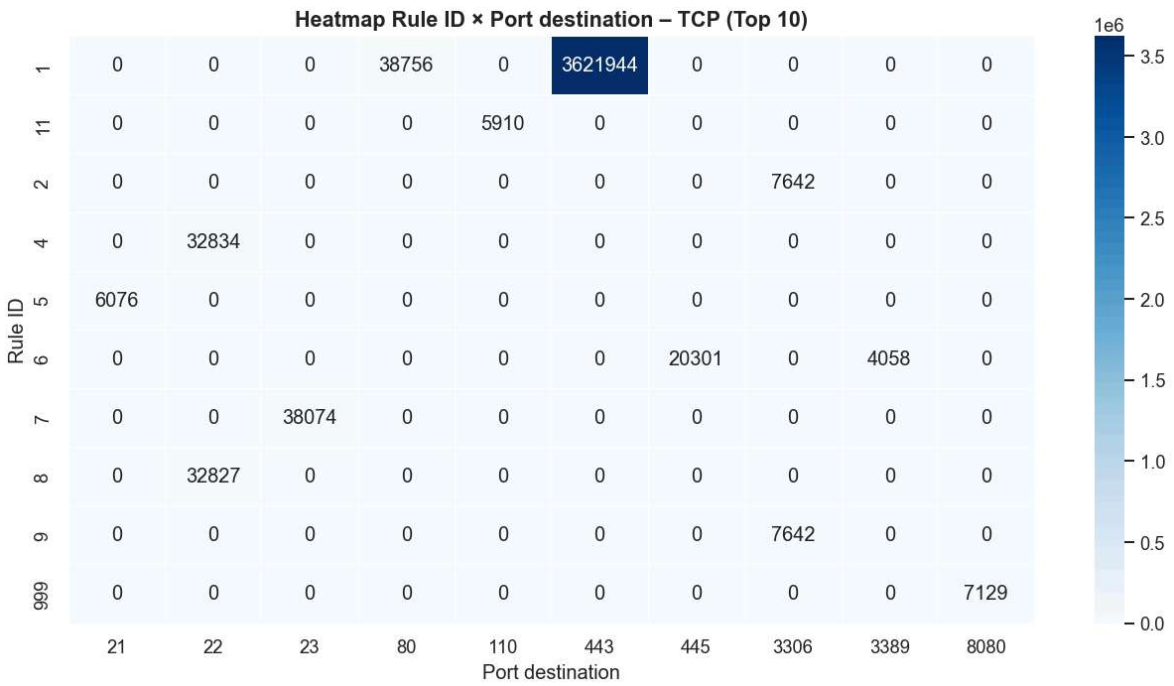
```
         Occurrences TCP
rule_id
1                3660700
999               755909
7                  38074
4                  32834
8                  32827
```

Chapitre 5 – Rapprochement Rule ID / Port destination / Action (TCP)

In [8]:
```
top_rules_tcp = df_tcp["rule_id"].value_counts().head(15).index
df_tcp_top = df_tcp[df_tcp["rule_id"].isin(top_rules_tcp)]
pivot_action = df_tcp_top.groupby(["rule_id", "action"]).size().unstack(fill_val

fig, ax = plt.subplots(figsize=(11, 6))
pivot_action.plot(kind="bar", ax=ax, color=PALETTE[:len(pivot_action.columns)],
ax.set_title("Rapprochement Rule ID / Action – TCP (Top 15 règles)", fontsize=13
ax.set_xlabel("Rule ID")
ax.set_ylabel("Occurrences")
ax.tick_params(axis="x", rotation=45)
ax.legend(title="Action")
plt.tight_layout()
plt.show()
```

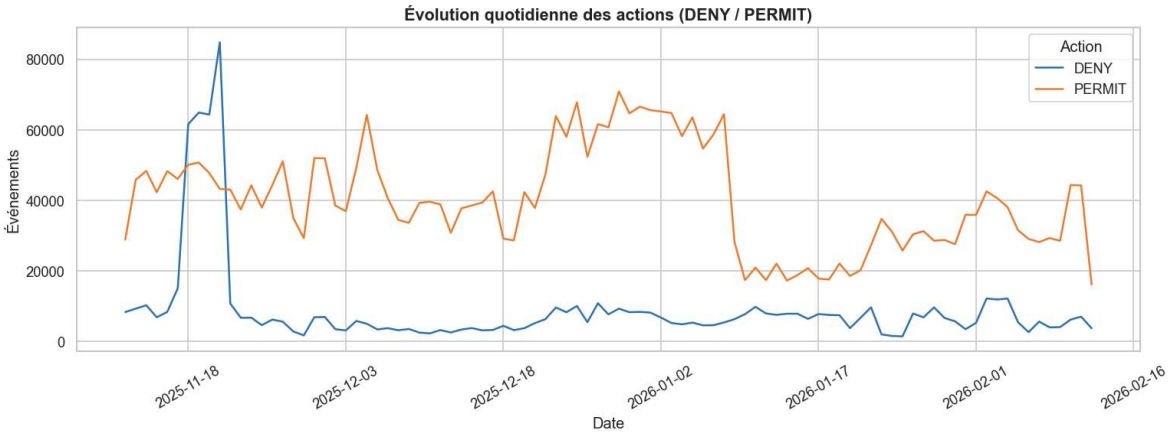


In [9]:
```
top_p = df_tcp["dst_port"].value_counts().head(10).index
df_heat = df_tcp[df_tcp["rule_id"].isin(top_rules_tcp) & df_tcp["dst_port"].isin
pivot_heat = df_heat.groupby(["rule_id", "dst_port"]).size().unstack(fill_value=

fig, ax = plt.subplots(figsize=(11, 6))
sns.heatmap(pivot_heat, annot=True, fmt="d", cmap="Blues", linewidths=0.5, ax=ax
ax.set_title("Heatmap Rule ID × Port destination – TCP (Top 10)", fontsize=13, f
ax.set_xlabel("Port destination")
ax.set_ylabel("Rule ID")
plt.tight_layout()
plt.show()
```



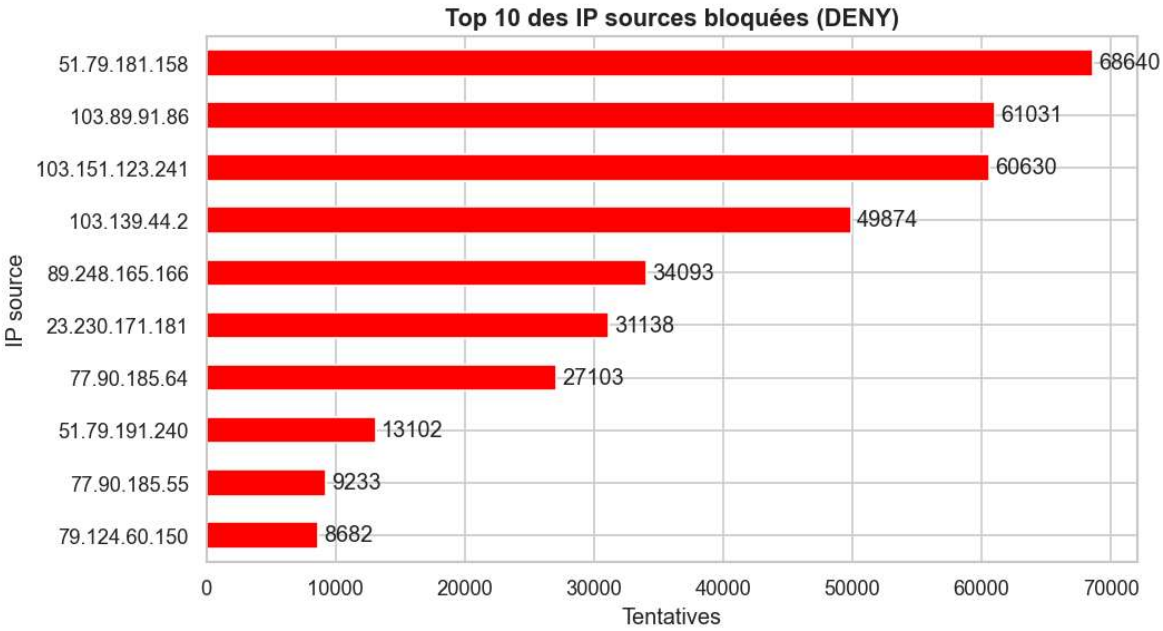
Chapitre 6 – Indicateurs de sécurité des Systèmes d'Information

In [10]:
```
# 6.1 – Évolution temporelle des actions
df_daily = df.groupby([df["timestamp"].dt.date, "action"]).size().unstack(fill_v

fig, ax = plt.subplots(figsize=(13, 5))
df_daily.plot(ax=ax, color=PALETTE[:len(df_daily.columns)], linewidth=1.5)
ax.set_title("Évolution quotidienne des actions (DENY / PERMIT)", fontsize=13, f
ax.set_xlabel("Date")
ax.set_ylabel("Événements")
ax.legend(title="Action")
ax.xaxis.set_major_locator(ticker.MaxNLocator(10))
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



In [11]:
```
# 6.2 – Top 10 IP sources bloquées
top_src = df[df["action"] == "DENY"]["src_ip"].value_counts().head(10)

fig, ax = plt.subplots(figsize=(9, 5))
top_src.plot(kind="barh", ax=ax, color=PALETTE[5])
ax.invert_yaxis()
```

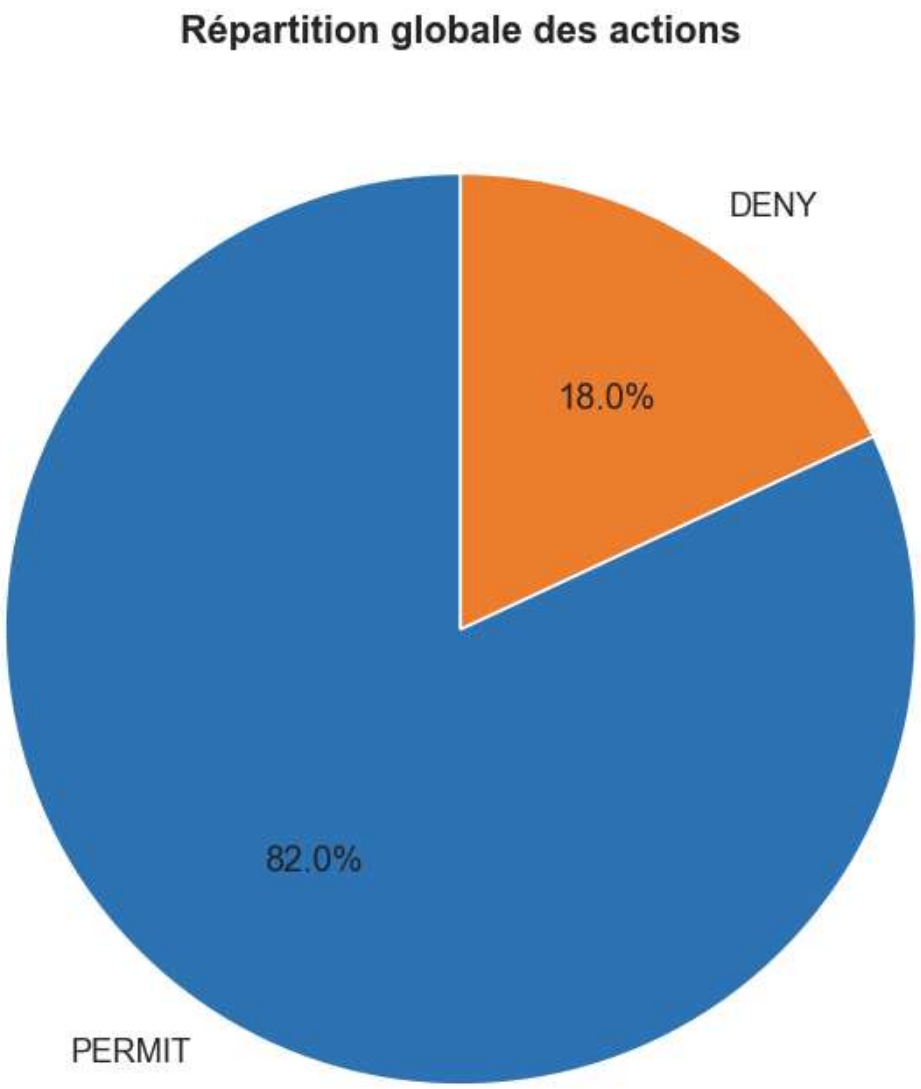
```
ax.set_title("Top 10 des IP sources bloquées (DENY)", fontsize=13, fontweight="b
ax.set_xlabel("Tentatives")
ax.set_ylabel("IP source")
ax.bar_label(ax.containers[0], fmt="%d", padding=3)
plt.tight_layout()
plt.show()

print(top_src.rename("Nb DENY").to_frame().to_string())
```




```
                 Nb DENY
src_ip
51.79.181.158      68640
103.89.91.86       61031
103.151.123.241    60630
103.139.44.2       49874
89.248.165.166     34093
23.230.171.181     31138
77.90.185.64       27103
51.79.191.240      13102
77.90.185.55        9233
79.124.60.150       8682
```

In [12]:

```
# 6.3 — Répartition globale des actions
action_counts = df["action"].value_counts()

fig, ax = plt.subplots(figsize=(6, 6))
ax.pie(action_counts, labels=action_counts.index, autopct="%1.1f%%",
       colors=PALETTE[:len(action_counts)], startangle=90)
ax.set_title("Répartition globale des actions", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

print(action_counts.rename("Occurrences").to_frame().to_string())
```




```
        Occurrences
action
PERMIT      3747555
DENY         825348
```